

Eng 4.4. SAST

Task 1 - Introduction

Static Application Security Testing (SAST), one of the many tools used to detect vulnerabilities during the development lifecycle of an application.

Task 2 - Code Review

Code reviews consist of looking at the source code of an application to search for possible vulnerabilities using a white box approach. By having access to the code, the reviewer can guarantee a greater coverage of the application's functionalities and lower the time required to find bugs.

Manual vs Automated Code Review

Manual code reviews have the advantage of a human evaluating the code, which allows for a thorough analysis and more precise results. However, since an application often has thousands and thousands of lines of code, the task can quickly become overwhelming for the reviewer, leading to some vulnerabilities being missed because of fatigue.

Automated tools excel at finding common vulnerabilities almost instantly, saving loads of time to the reviewer. Automated tools will also perform consistently, no matter the size of the code base, so they won't miss vulnerabilities as a human could do, as long as they have predefined rules to match them.

Answer the questions below

| Are automated code reviews a substitute for manual reviewing?

no

| What type of code review will run faster?

Automated

| What type of code review will be more thorough?

Manual

Task 3 - Manual Code Review

Before jumping into SAST tools, we will look at how manual code reviewing is usually performed, as this will make it easier to understand how SAST tools look at code. During this task, our goal will be to find SQL injection vulnerabilities in the source code of a simple application. The application has other vulnerabilities, but we will focus on a single type of vulnerability to understand how traditional code reviews are done and how those steps map to the analysis techniques used by SAST tools. For this task, the code we will be using can be found in `/home/ubuntu/Desktop/simple-webapp/`

Searching for Insecure Functions

Database Engine	Function
MySQL	<code>mysqli_query()</code> <code>mysql_query()</code> <code>mysqli_prepare()</code> <code>query()</code> <code>prepare()</code>

For example, to search for instances of `mysqli_query()`, go to the project's base directory and run the following command:

```
user@machine$ cd /home/ubuntu/Desktop/simple-webapp/html/

user@machine$ grep -r -n 'mysqli_query('
db.php:18: $result = mysqli_query($conn, $query);
```

The `-r` option tells `grep` to recursively search all files under the current directory, and the `-n` option indicates that we want `grep` to tell us the number of the line where the pattern was found. In the above terminal output, we can see that a single instance of `mysqli_query()` was found on line `18`. We have identified a line that could lead to SQL injections. However, just by looking at that line, we can't determine whether a vulnerability is present.

Understanding the Context: Let's open `db.php` to analyse the context around the `mysqli_query()` function to see if we get a better idea of how it is used:

```
function db_query($conn, $query){
    $result = mysqli_query($conn, $query);
```

```
    return $result;
}
```

Here we can see that `mysqli_query()` is wrapped into the `db_query()` function, and that the `$query` parameter is passed directly without modification. It is very common for functions to be nested into other functions, so simply analysing the local context of a function is sometimes not enough to determine if a vulnerability is present. We now need to trace the uses of the `db_query()` function throughout our code to identify potential vulnerabilities.

Tracing User Inputs to Potentially Vulnerable Functions: We can use `grep` again to search for uses of `db_query()`:

```
user@machine$ grep -rn 'db_query('
hidden-panel.php:7:$result = db_query($conn, $sql);
hidden-panel.php:20:$result2 = db_query($conn, $sql2);
hidden-panel.php:23:$result3 = db_query($conn, $sql3);
db.php:17:function db_query($conn, $query){
```

Here we get three uses for `db_query()` on `hidden-panel.php`. Once again, we can analyse the context of each call, starting with the call on line 7:

```
$sql = "SELECT id, firstname, lastname FROM MyGuests WHERE id
=".$_GET['guest_id'];
$result = db_query($conn, $sql);
```

Congrats on finding an SQL injection! Whatever is passed in the `guest_id` parameter via the GET method will be concatenated to a raw SQL query without any input sanitisation, enabling the attacker to change the query.

Local File Inclusion (LFI) attacks are made possible by the misuse of one of the following functions in PHP: `require()` `include()` `require_once()` `include_once()`
Answer the following questions using `grep` to search for LFI vulnerabilities only on the `.php` files in the `html/` directory of the `simple-webapp` project.

```
ubuntu@tryhackme:~/Desktop/simple-webapp/html$ grep -rn 'include('
login.php:35:<?php include('navbar.php'); ?>
nospiders-gallery.php:16:<?php include('navbar.php'); ?>
hidden-panel.php:2:include("db.php");
hidden-panel.php:44:<?php include('navbar.php'); ?>
view.php:16:<?php include('navbar.php'); ?>
view.php:22:include('./gallery-files/'.$_GET['img']);
aboutme.php:16:<?php include('navbar.php'); ?>
cowsay.php:24:<?php include('navbar.php'); ?>
index.php:16:<?php include('navbar.php'); ?>
```

"This grep command recursively searches all files for occurrences of include(and is commonly used in code review and security audits to identify file inclusion patterns."

Answer the questions below

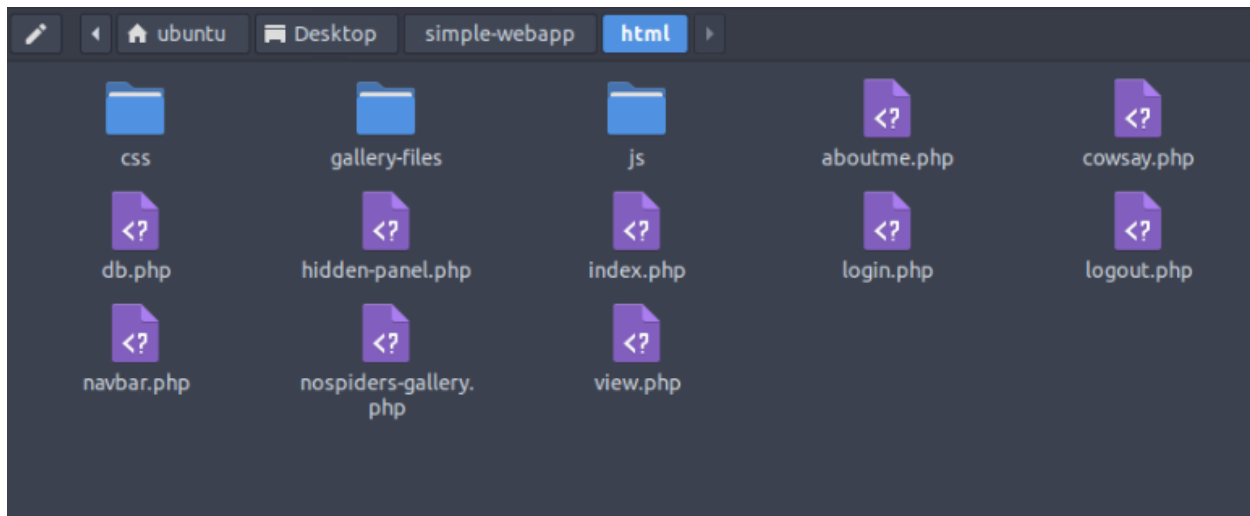
Which of the mentioned functions is used in the project?

include()

How many instances of the function found in question 2 exist in your project's code?

9

Only one of the function's instances is vulnerable to LFI. Remember that for LFI to be present, the attacker must be able to manipulate a part of what is sent to the vulnerable function. The vulnerable instance must contain some reference to a GET or POST parameter or other manipulable inputs. What file contains the vulnerable instance? view.php



```
ubuntu@tryhackme:~/Desktop/simple-webapp/html$ grep -rn 'include('
login.php:35:<?php include('navbar.php'); ?>
nospiders-gallery.php:16:<?php include('navbar.php'); ?>
hidden-panel.php:2:include("db.php");
hidden-panel.php:44:<?php include('navbar.php'); ?>
view.php:16:<?php include('navbar.php'); ?>
view.php:22:include('./gallery-files/'.$_GET['img']);
aboutme.php:16:<?php include('navbar.php'); ?>
cowsay.php:24:<?php include('navbar.php'); ?>
index.php:16:<?php include('navbar.php'); ?>
ubuntu@tryhackme:~/Desktop/simple-webapp/html$ cat view.php
<!doctype html>
<html lang="en">

<head>
    <title>ASCII Art</title>
    <!-- Required meta tags -->
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

    <!-- Bootstrap CSS -->
    <link rel="stylesheet" href="css/bootstrap.min.css">
    <link rel="stylesheet" href="css/main.css">
</head>

<body>
<?php include('navbar.php'); ?>
```

What line in the file found on the previous question is vulnerable to LFI?

22

Task 4 - Automated Code Review

Static Application Security Testing (SAST) refers to using automated tools for code analysis. SAST complements other techniques, such as Dynamic Application Security Testing (DAST) or Software Composition Analysis (SCA), to provide a holistic approach to application security during the development lifecycle.

Pros: • It doesn't require a running instance of the target application. • It provides great coverage of the application's functionality. • It runs fast as opposed to other dynamic techniques. • SAST tools report exactly where vulnerabilities are in the code. • Easy to integrate into your CI/CD pipeline.	Cons: • The source code of an application is not always available (third-party apps). • Prone to false positives. • Can't identify vulnerabilities that are dynamic in nature. • SAST tools are mostly language-specific. They can only check languages they know.
--	---

While every SAST tool is different, most of them will perform two main tasks:

1. **Transform the code into an abstract model:** SAST tools usually ingest the source code and produce an abstract representation for further analysis. Most SAST tools will represent the code using Abstract Syntax Trees (AST), but some tools may have other equivalent proprietary structures. This allows for easier code analysis in a way that is independent of the programming language in use.
2. **Analyse the abstract model for security issues:** Different analysis techniques will be used to search for potential vulnerabilities in the code model.

Answer the questions below

| Does SAST require a running instance of the application for analysis?

no

| What kind of analysis would likely flag dead code segments?

structural analysis

| What kind of analysis would likely detect flaws in configuration files?

configuration analysis

| What kind of analysis is similar to grepping the code in search of flaws?

semantic analysis

Task 5 - Rechecking our Application with SAST Tools

Psalm (PHP Static Analysis Linting Machine), a simple tool for analysing PHP code. By default, Psalm will only run some structural analysis over the code and show us programming errors that need our attention. In the previous example, Psalm reports that the condition on the `if` clause is always `false`. In this case, the programmer mistakenly used the assignment operator (`=`) instead of the comparison operator (`==`). Since the assignment operator always returns the assigned value (`0` in this case), the condition will always evaluate as false (`0` will be automatically cast to `false` in PHP). Psalm also offers the possibility to run dataflow analysis on our code using the `--taint-analysis` flag. The output for this command will be much more interesting as it will pinpoint possible security issues. The output of the command will look like this:

```
user@machine$ cd /home/ubuntu/Desktop/simple-webapp/
user@machine$ ./vendor/bin/psalm --no-cache --taint-analysis

ERROR: TaintedInclude - html/view.php:22:9 - Detected tainted
code passed to include or similar (see https://psalm.dev/251)
  $_GET
    <no known location>

  $_GET['img'] - html/view.php:22:28
include('./gallery-files/'.$_GET['img']);

concat - html/view.php:22:9
include('./gallery-files/'.$_GET['img']);
```

For each error reported, Psalm will show you a complete trace of the data flow from the source (`$_GET`) to the sink function (`include()`). Here we have a typical

Local File Inclusion (LFI) vulnerability, as the GET parameter `img` is concatenated directly into the filename passed to the `include()` function.

- **False positives:** The tool reports on a vulnerability that isn't present in the code.
- **False negatives:** The tool doesn't report a vulnerability that is present in the code.

Answer the questions below

What type of error occurs when the tool reports on a vulnerability that isn't present in the code?

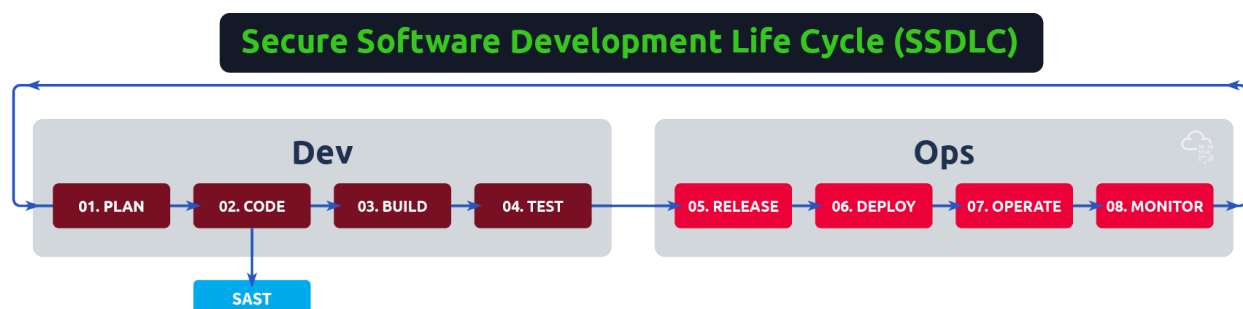
false positive

How many errors are reported after annotating the code as instructed in this task and re-running Psalm?

9

Task 6 - SAST in the Development Cycle

SAST is one of the first tools to appear during the development lifecycle. It is often implemented during the coding stage, as there's no need to have a functional application to use it.



Depending on each case, SAST can be implemented in one of the following ways:

- **CI/CD integration:** Each time a pull request or a merge is made, SAST tools will check the code for vulnerabilities. Checking pull requests ensures that the

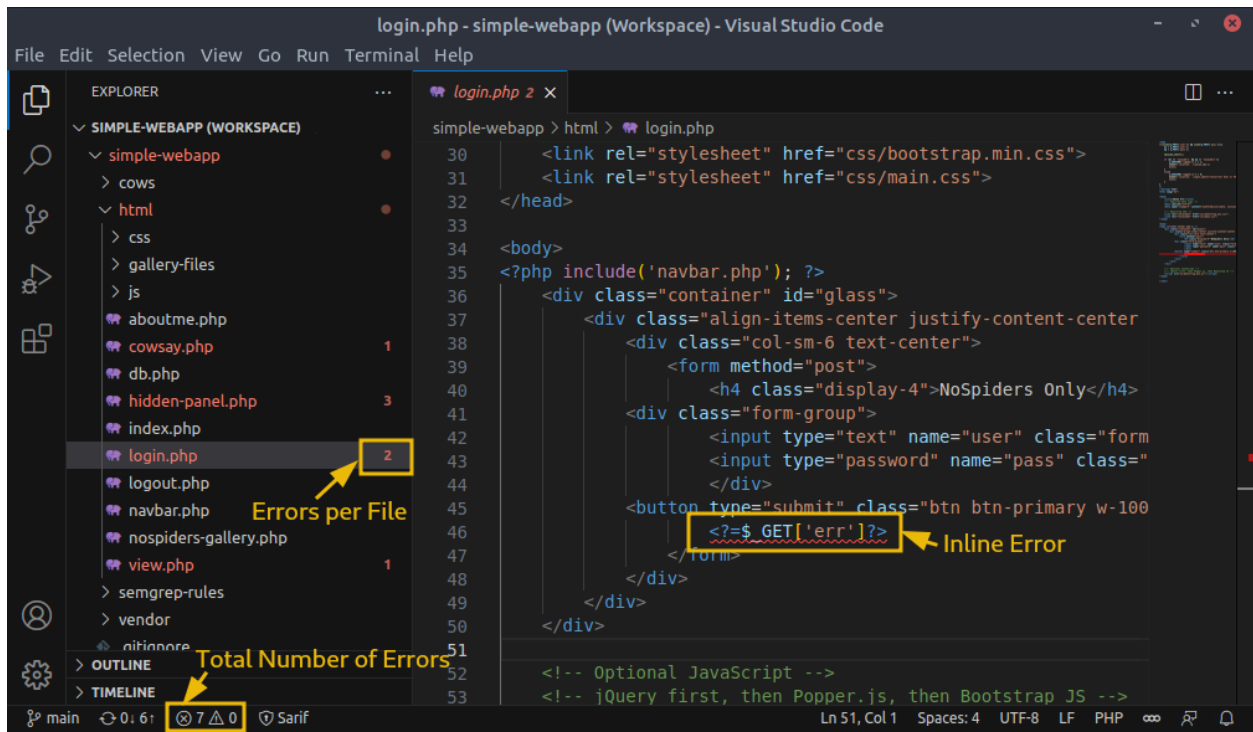
code that makes it to merges has undergone at least a basic security check. On some occasions, instead of checking every single pull request, executing SAST scans only at merges may help prevent slowdowns in the development pipeline, as it avoids making all developers wait for full SAST scans.

- **IDE integration:** SAST tools can be integrated into the developers' favourite IDEs instead of waiting for a pull request or merge to occur. This way, the code can be fixed as early as possible, saving time further ahead in the project.

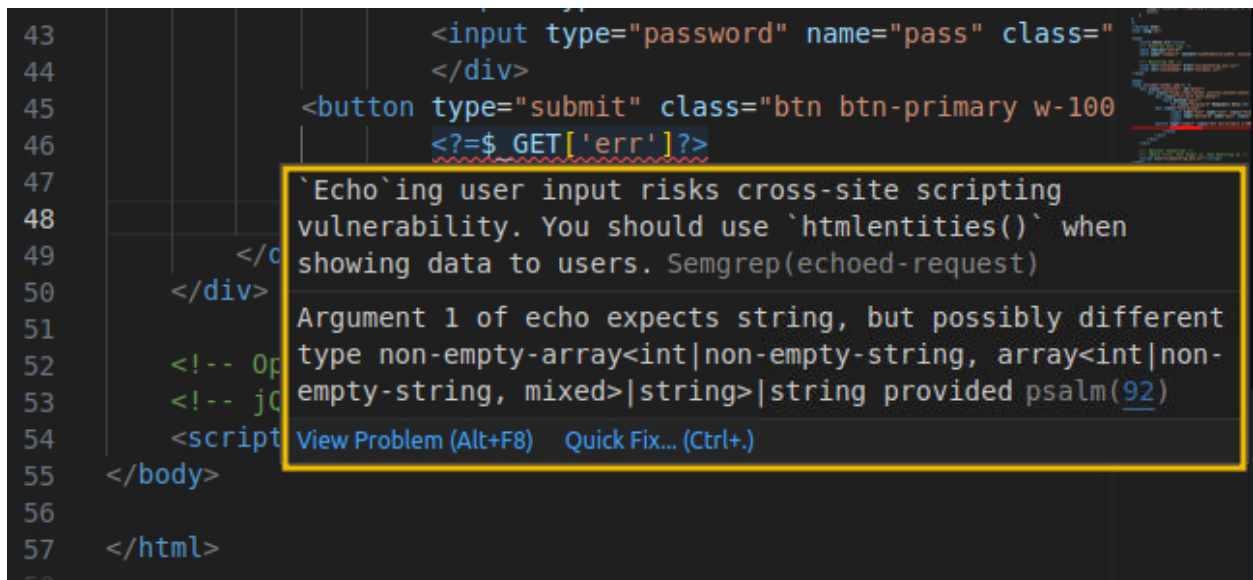
Integrating SAST in IDEs

- **Psalm:** The tool we've been using supports IDE integration by installing the Psalm plugin into VS Code directly from the [Visual Studio Marketplace \(opens in new tab\)](#). This plugin will check anything you type in real-time and show you the same alerts as the console version directly into your code. Taint analysis won't be available, so you'll only see the structural problems reported on a basic Psalm analysis.
- **Semgrep:** Yet another SAST tool that can be installed into VS Code directly from the [Visual Studio Marketplace \(opens in new tab\)](#). Just as with Psalm, it will show inline alerts directly in your code. Semgrep even allows us to build custom rules if needed. You can check the rules that are loaded for this project on the `semgrep-rules` directory inside the project's directory. The specifics on how to build Semgrep rules will be left for a future room, so don't worry about them now. You can check the rules in the directory, but this won't be required for this room.

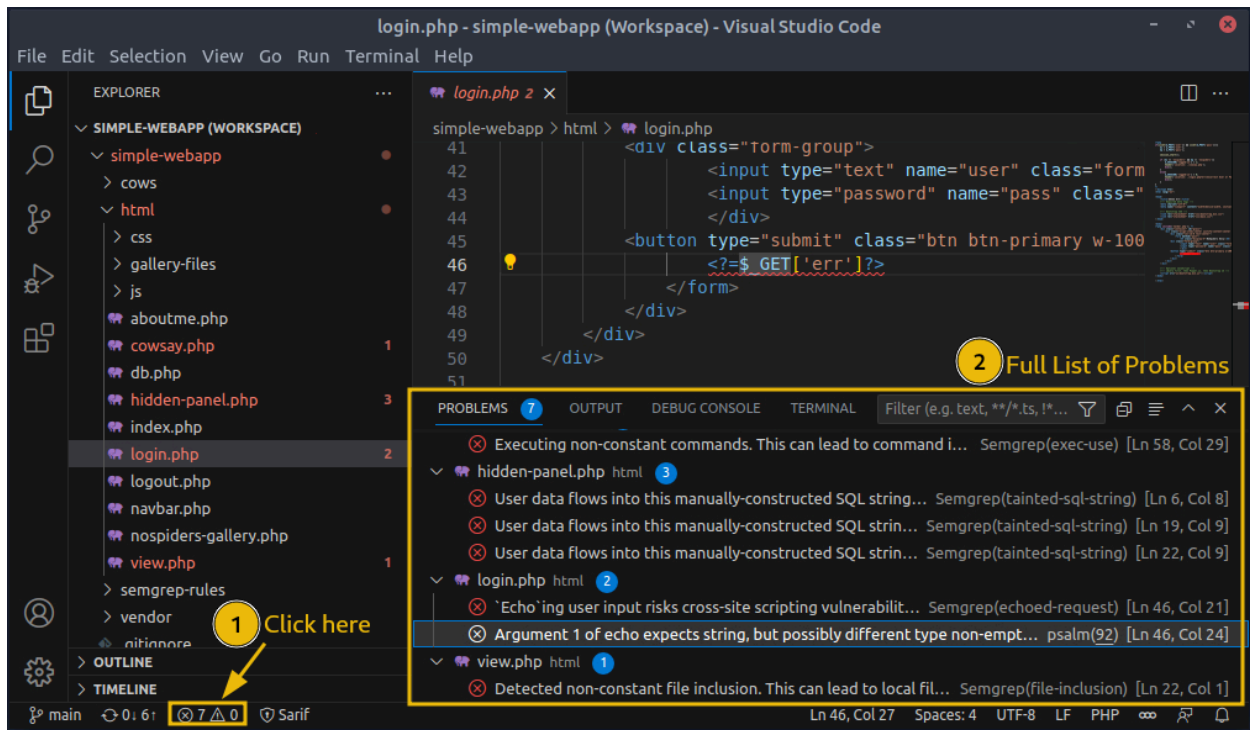
Both tools will work similarly by showing any problems detected directly inline in your code. The only notable difference is that Semgrep will run when you start VS Code and will show you the problems it detects on all of the files, while Psalm will only show you issues related to the file you are currently editing. The plugins will add the following information to your IDE screen:



For each line where issues are detected, you can get additional information by hovering the mouse pointer over it. Here's what you would get by hovering over line 46 of `login.php` :



You can also check the complete list of issues by clicking the Errors indicator at the bottom of VS Code's screen:



Having your IDE check for problems allows you to produce cleaner code as a developer, contributing significantly to the overall security of your final application.

Answer the questions below

How many problems in total are detected by Semgrep in this project?

27

How many problems are detected in the showrecipe.inc.php file?

8 Open showrecipe.inc.php. There are two types of problems being reported by Semgrep in this file. One is identified as "tainted-sql-string" and refers to possible SQL injections.

What other problem identifier is reported by Semgrep in this file?

echoed-request (Write the id reported by Semgrep)

What type of vulnerability is associated with the problem identifier on the previous question?

cross-site scripting

Task 7 - Conclusion

SAST is one of the many techniques we can use to improve the security of our applications while they are still being developed. We have shown how to use Psalm, one of the many available SAST tools and how much time it saves us compared to manual reviews. As with any other automated tool, it is essential to validate the results manually, as false positives may be reported.

SAST (Static Application Security Testing) is a security testing technique that analyzes an application's **source code, bytecode, or binary code without executing it** to find security vulnerabilities. It is also called **white-box testing** because it examines the internal structure of the application.

How SAST Works

1. Developer writes code
2. SAST tool scans the code (before running the app)
3. It analyzes:
 - Data flow
 - Control flow
 - Code patterns
4. It compares against known vulnerability rules
5. It generates a report with issues and severity

What SAST Detects

- SQL Injection
- Cross-Site Scripting (XSS)
- Hardcoded credentials
- Command injection
- Insecure cryptography
- Buffer overflows

- Path traversal issues
 - Insecure API usage
-

Popular SAST Tools

- GitHub – Deep code analysis for vulnerabilities
- Sonar – Code quality + security scanning
- Semgrep – Lightweight rule-based scanning
- Checkmarx – Enterprise SAST solution
- Fortinet – Advanced enterprise static analysis
- Synopsys – High-precision code analysis
- Veracode – Cloud-based security scanning

Benefits of SAST

- Finds vulnerabilities early (shift-left security)
- No need to run the application
- Easy integration into CI/CD pipelines
- Reduces cost of fixing bugs
- Improves secure coding practices

Limitations of SAST

- May produce false positives
 - Cannot detect runtime-only issues
 - Limited visibility into external dependencies
 - Requires access to source code
-

SAST vs DAST

Feature	SAST	DAST
Type	Static (code analysis)	Dynamic (running app)
Code required	Yes	No

Feature	SAST	DAST
Stage	Development	Testing / staging
Approach	White-box	Black-box
Detects runtime issues	No	Yes

Interview Answer

"SAST is a static code analysis technique used to identify security vulnerabilities in source code without executing the application. It is typically used early in the SDLC to detect issues like SQL injection, XSS, and insecure coding patterns. It helps developers fix vulnerabilities before deployment and integrates well with CI/CD pipelines."

One-line definition

SAST is a white-box security testing method that scans source code to detect vulnerabilities before the application is run.

IDE (Integrated Development Environment) is a software application that provides developers with a complete set of tools to write, test, debug, and manage code in one place.

What an IDE Includes: An IDE typically combines several tools:

-  **Code editor** – Write and edit source code
-  **Compiler / interpreter** – Convert code into executable programs
-  **Debugger** – Find and fix errors in code
-  **Project manager** – Organize files and dependencies
-  **Code completion & suggestions** – Auto-suggest code as you type
-  **Build & run tools** – Execute applications directly

Why IDEs Are Important

- Increase developer productivity
- Reduce coding errors with real-time feedback
- Simplify debugging and testing

- Help manage large projects efficiently
- Provide language-specific support and frameworks

Popular IDEs

- Visual Studio Code – Lightweight, highly extensible, supports many languages
- IntelliJ IDEA – Powerful IDE for Java development
- PyCharm – Designed for Python development
- Eclipse – Widely used for Java and enterprise applications
- Visual Studio – Full-featured IDE for Windows development
- Android Studio – Built for Android apps

IDE vs Text Editor

Feature	IDE	Text Editor
Tools included	Compiler, debugger, build tools	Basic editing only
Complexity	Heavy / feature-rich	Lightweight
Best for	Large projects	Quick edits or scripting
Examples	IntelliJ, Visual Studio	Notepad++, basic VS Code setup

An IDE is an all-in-one software workspace that helps developers write, test, and debug code more efficiently.